

HOMWORK 5-6

DATA STRUCTURES *

10-607 COMPUTATIONAL FOUNDATIONS FOR MACHINE LEARNING

START HERE: Instructions

- **Collaboration Policy:** Please read the collaboration policy in the syllabus.
- **Late Submission Policy:** See the late submission policy in the syllabus.
- **Submitting your work:** You will use Gradescope to submit answers to all questions.
 - **Written:** For written problems such as short answer, multiple choice, derivations, proofs, or plots, please use the provided template. Submissions can be handwritten onto the template, but should be labeled and clearly legible. If your writing is not legible, you will not be awarded marks. Alternatively, submissions can be written in $\text{\LaTeX}$. Each derivation/proof should be completed in the boxes provided. To receive full credit, you are responsible for ensuring that your submission contains exactly the same number of pages and the same alignment as our PDF template.
 - **Latex Template:** <https://www.overleaf.com/read/fspghmbfbsw#8a6030>

Question	Points
Vector Programming	18
Tree Programming	8
Graphs	11
Divide and Conquer	14
Total:	51

*Compiled on Wednesday 19th November, 2025 at 21:27

Instructions for Specific Problem Types

For “Select One” questions, please fill in the appropriate bubble completely:

Select One: Who taught this course?

- ☒ Bryan Wilder
- ☐ Marie Curie
- ☐ Noam Chomsky

If you need to change your answer, you may cross out the previous answer and bubble in the new answer:

Select One: Who taught this course?

- ☒ Bryan Wilder
- ☐ Marie Curie
- ☒ Noam Chomsky

For “Select all that apply” questions, please fill in all appropriate squares completely:

Select all that apply: Which are scientists?

- ☒ Stephen Hawking
- ☒ Albert Einstein
- ☒ Isaac Newton
- ☐ I don't know

Again, if you need to change your answer, you may cross out the previous answer(s) and bubble in the new answer(s):

Select all that apply: Which are scientists?

- ☒ Stephen Hawking
- ☒ Albert Einstein
- ☒ Isaac Newton
- ☐ I don't know

For questions where you must fill in a blank, please make sure your final answer is fully included in the given space. You may cross out answers or parts of answers, but the final answer must still be within the given space.

Fill in the blank: What is the course number?

10-606

10-6067

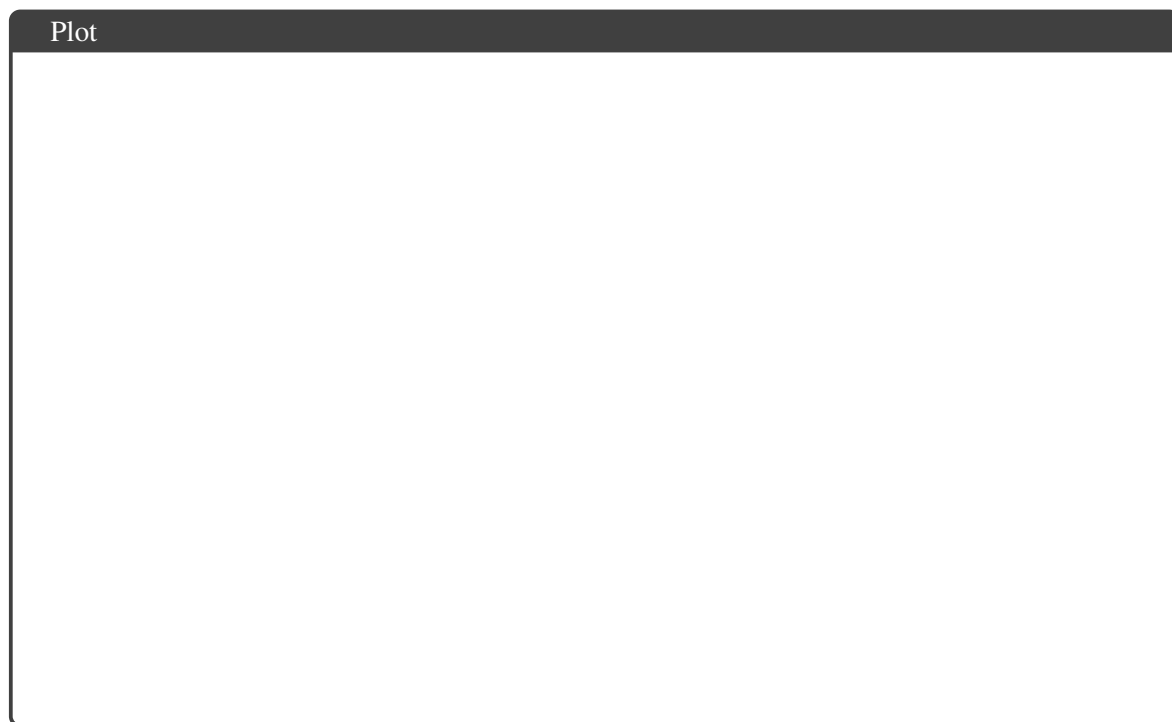
1 Vector Programming (18 points)

In this question, we will study the computational efficiency of running a linear regression model which makes predictions $f(x) = \beta^T x$ for an input $x \in \mathbb{R}^n$ and coefficient vector $\beta \in \mathbb{R}^n$. That is, each call to f requires computing a vector dot product between x and β . In the high dimensional setting, n (the number of features) becomes large and this operation might be computationally expensive. We'll analyze the difference in performance between implementations using for loops and those using NumPy. We'll also look at sparse representations of vectors and their effect on performance. The following Jupyter notebook contains the instructions for implementing the three dot products functions and running three performance experiments:

https://drive.google.com/file/d/1Bxi6cL0aqpcLUKud_o2LYfWgbCiOV-9q

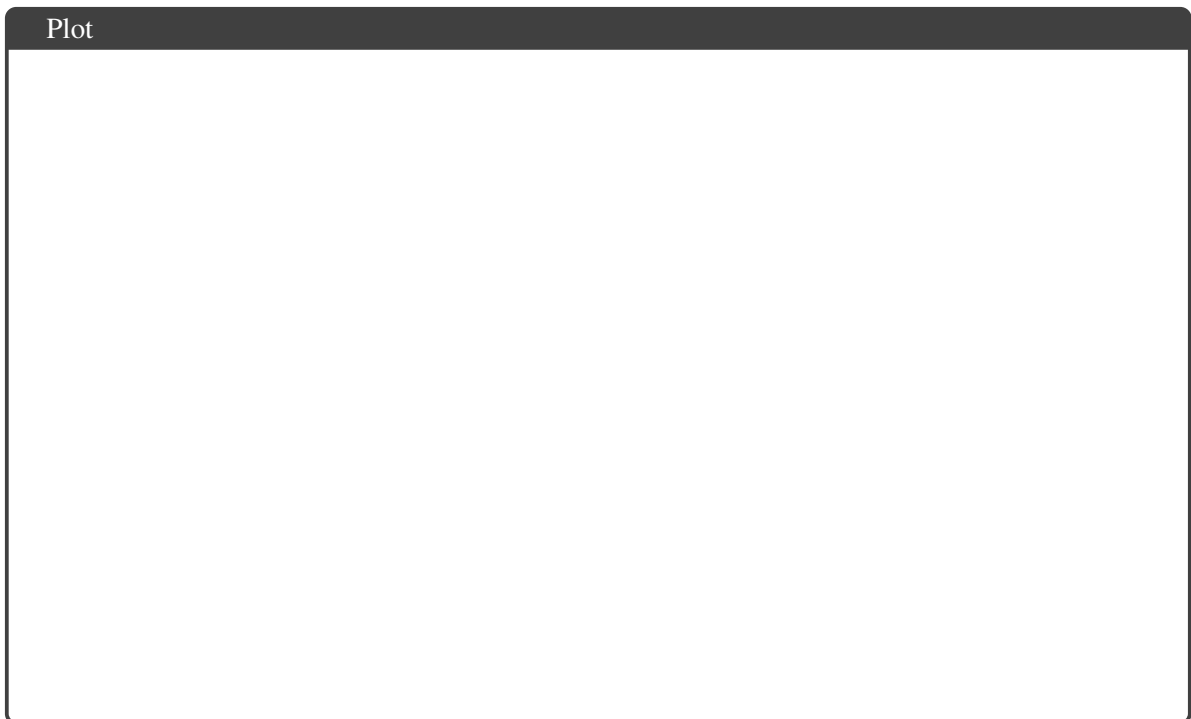
Note: To make it more likely that we are all running our code in similar environments, if possible, please use Google Colab to run the performance experiments in the above file. After running the performance experiments, provide your plots and answer the questions below.

1. (3 points) Experiment 1: Make a plot that compares the empirical runtime of your NumPy implementation to the length of the vectors. Follow the detailed instructions for `Experiment 1` in the provided code and submit your plot below. Use the plot to provide an *empirical* estimate of the runtime complexity.



2. (1 point) What is the (simplest and tightest) time complexity that you would expect for a dot product implementation with respect to the vector length N ?
☐ $O(1)$ ☐ $O(\log N)$ ☐ $O(N)$ ☐ $O(N \log N)$ ☐ $O(N^2)$
3. (2 points) Do the empirical results in your plot seem to support this complexity? Explain your reasoning in one sentence. (Note that we, of course, can't measure the actual complexity from just these empirical results with a finite N).

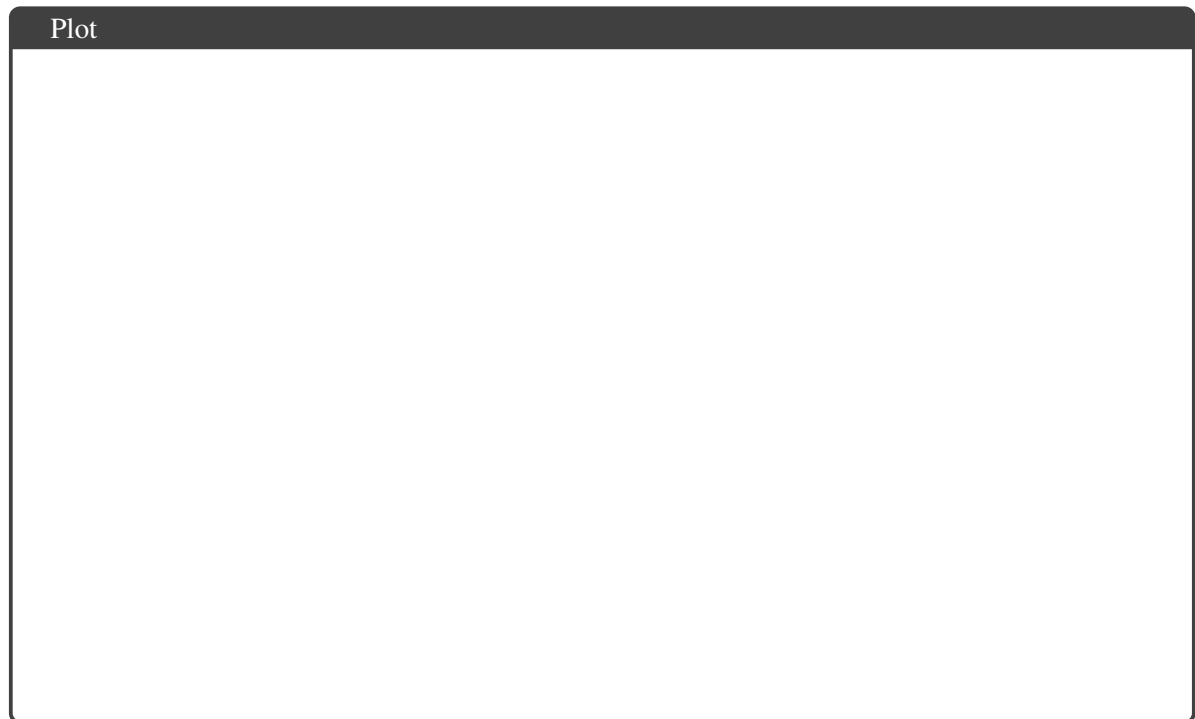
4. (3 points) Experiment 2: Plot the runtime of the for-loop and NumPy implementations as the length of the vectors changes. Follow the detailed instructions for `Experiment 2` in the provided code and submit your plot below.



5. (2 points) Do the empirical results for Experiments 1 and 2 support the claim that for-loop and NumPy implementations are in the same complexity class? Explain your reasoning in one or two sentences.

(Again, we, of course, can't measure the actual complexity from just these empirical results with a finite N).

6. (3 points) Experiment 3: Plot the runtime of the NumPy and sparse-for-loop implementations as the proportion of nonzero entries changes. Follow the detailed instructions for Experiment 3 in the provided code and submit your plot below.



7. (2 points) What is the time complexity for the sparse-for-loop dot product implementation with respect to the vector length N and the number of nonzeros K ? Give your answer in big-O notation with the simplest and tightest bound. Note: this is not an empirical estimate, you should analyze your code.

8. (2 points) Based on your Experiment 3 plot, give an estimate for the density of nonzeros, K/N , below which it may be more efficient to use the sparse-for-loop implementation. Give your answer as a decimal value between 0.0 and 1.0 (not as a fraction).

Programming Submission

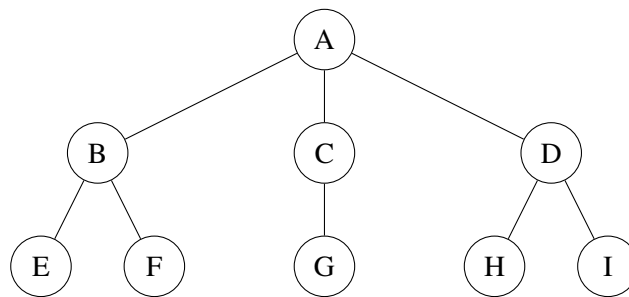
Please submit the file `vector.ipynb` (in addition to `tree.py`) directly to Gradescope under the HW5 (programming) assignment. Note, there are no autograder tests associated with this submitted file on Gradescope. We will manually grade the code for your dot product implementations and your experiments.

2 Tree Programming (8 points)

All your code for this question should be in a Python file named `tree.py`.

First implement the following three data structures.

- In Python, implement a `Stack` data structure with two functions (`push` and `pop`).
- Implement a `Queue` data structure with two functions (`enqueue` and `dequeue`).
- Implement a `Tree` data structure. One way to do this is to use a Python dictionary where the condition at a node can be used as a string key to traverse the children of the node. A more object-oriented way is to create a `TreeNode` class that stores the label of the node and a list of children, which themselves are also `TreeNodes`. Using your tree implementation, create the following tree in Python code:

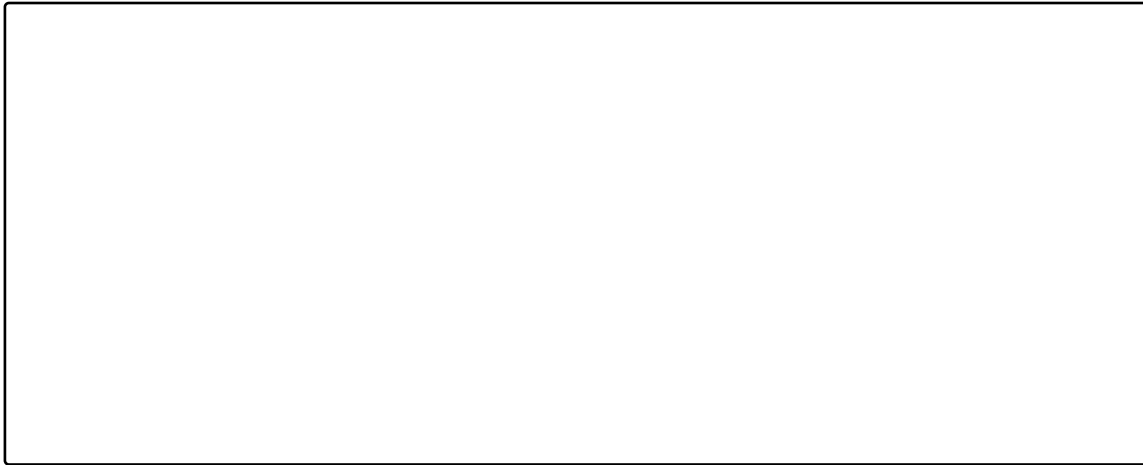


1. (4 points) Use depth first search to traverse the tree and print out each node in the order visited. **Make sure to use either the stack or queue defined in part 1 of this question as part of your implementation.** Put all of your code in the function `def tree_dfs()`.

Include a legible screenshot of your console output here.

2. (4 points) Use breadth first search to traverse the tree and print out each node in the order visited. **Make sure to use either the stack or queue defined in part 1 of this question as part of your implementation.** Put all of your code in the function `def tree_bfs()`.

Include a legible screenshot of your console output here.

**Instructions for submitting code**

Please submit the file `tree.py` (in addition to `vector.ipynb`) directly to Gradescope under the HW5-6 (programming) assignment. Note, there are no autograder tests associated with this submitted file on Gradescope. We will manually grade the code for your tree and search implementations.

3 Graphs (11 points)

Consider the following pseudocode for a `Puzzle` class. The name of the class is chosen to indicate that in the following questions you will need to *puzzle* through what this class actually does. The `__init__(G, s)` method takes as arguments a directed graph G and a vertex s . If v is a vertex in G , then `G.get_adjacent_nodes(v)` returns an iterable collection of all vertices that are adjacent to v in G .

```
class Puzzle:
    def __init__(self, G, s):
        self.tagged = {}
        puzzle(G, s)

    def puzzle(G, v):
        self.tagged[v] = True
        for w in G.get_adjacent_nodes(v):
            if w not in self.tagged:
                puzzle(G, w)

    def tagged(v):
        return self.tagged[v]
```

1. (5 points) Suppose we create a `Puzzle` object with a directed graph G and vertex s . Give a short description of what the method `tagged` returns for an input vertex v if the object is initialized with a directed graph G and a vertex s ?

Solution

2. (2 points) Suppose that we represent a directed graph using the adjacency-lists representation. When we create a new `Puzzle` object, what is the runtime complexity of this action *in the worst case*?
☐ $O(1)$ ☐ $O(E)$ ☐ $O(V)$ ☐ $O(E + V)$ ☐ $O(V^2)$ ☐ $O(E^2)$
3. (2 points) Suppose that we represent a directed graph using the adjacency-lists representation. When we create a new `Puzzle` object, what is the runtime complexity of this action *in the best case*?
☐ $O(1)$ ☐ $O(E)$ ☐ $O(V)$ ☐ $O(E + V)$ ☐ $O(V^2)$ ☐ $O(E^2)$
4. (2 points) To store sparse graphs, which form would we prefer to use?
☐ Adjacency matrix
☐ Adjacency list

4 Divide and Conquer (14 points)

Consider the three following divide and conquer algorithms: (1) Mergesort, (2) Quicksort (if partitioning always divides the array in half), (3) Binary search in a sorted array.

1. (3 points) For each algorithm, write down a recurrence relation showing how $T(N)$, the running time on an instance of size N , depends on the running time of a smaller instance. Use c to denote costs incurred from constant time operations.

Solution

2. (3 points) For each algorithm, write down the corresponding runtime for each $T(N)$ (e.g., $T(N) \sim cN^2$)

Solution

3. (4 points) Given the following list $[7, 3, 10, 2, 1, 5, 2]$, draw out how the Mergesort algorithm would process the list (i.e., use boxes to split up the list and denote subroutines that are called in the algorithm)

Solution

4. (4 points) Given the following list $[7, 3, 10, 2, 1, 5, 2]$, draw out how the Quicksort algorithm would process the list (i.e., use boxes to split up the list and denote subroutines that are called in the algorithm).

Solution

5 Collaboration Questions

After you have completed all other components of this assignment, report your answers to these questions regarding the collaboration policy. Details of the policy can be found in the syllabus.

1. Did you receive any help whatsoever from anyone in solving this assignment? If so, include full details.
2. Did you give any help whatsoever to anyone in solving this assignment? If so, include full details.
3. Did you find or come across code that implements any part of this assignment? If so, include full details.

Your Answer